

Technical Report — v6

Number Embedding Representation

A Framework for Information-Preserving Numerical Embeddings

Bridging Mathematical Numerical Structure and Neural Vector Representations

February 2026 | Pure NumPy Implementation | 128-Dimensional Embedding Space

Table of Contents

1. Executive Summary
 2. Problem Statement
 3. Theoretical Framework
 4. Implementation Architecture
 5. v2 Changes — Encoder and Training Fixes
 6. v3 Changes — Stability, Tooling, and Loss Redesign
 7. Failure Analysis: 30k Softplus Run
 8. v4 Changes — Ceiling and Sign Fixes
 9. v4 Results and Remaining Issues
 10. v5 Changes — Multi-Objective Loss and Wider Clip
 11. v5 Results and Diagnosis
 12. v6 Changes — Distribution Matching, Log-Magnitude Loss, Extended Training
 13. v6 Results
 14. Application Analysis: Number Embeddings for LLMs
 15. Fundamental Limits of Learned Decoding
 16. Recommendations and Future Work
 17. Conclusion
 18. Appendix A: Design Discussion — v6 Results Analysis
 19. Appendix B: Design Discussion — Number Embeddings for LLMs
 20. Appendix C: Design Discussion — End-to-End Numerical Processing
-

1. Executive Summary

This report presents the full iterative development of the **Number Embedding Representation Framework**, now at v6 of the implementation. The framework embeds scalar numerical values into high-dimensional vector spaces while preserving five mathematical properties: uniqueness, continuity, reversibility, expressiveness, and model compatibility.

Development proceeded through six engineering iterations, each identifying and correcting a distinct class of failure:

- **v1** → **v2**: Fixed encoder magnitude destruction (LayerNorm erased scale), incorrect LayerNorm backward, and biased training distribution. Result: 20/22 → 22/22 tests at 12k

steps.

- **v2 → v3:** Added training stability infrastructure (gradient norm clipping, LR warmup), transitioned to log-space loss, added argparse and file logging, extended to 30k steps.
 - **v3 softplus experiment:** Replacing `exp` with `softplus` introduced a hard ceiling at magnitude ≈ 8 , causing a loss plateau at ~ 3.0 and sign confusion near zero.
 - **v3 → v4:** Reverted to `exp(clip(z, -8, 8))` and replaced pure log-space loss with **signed-log loss** to fix sign ambiguity. Fixed a critical gradient sign bug. Result: loss 0.000015 at 30k, 22/22 tests, but large numbers (100k $\rightarrow$ 2960) and near-zero sign still problematic.
 - **v4 → v5:** Added BCE sign loss, relative MSE (phase 2), widened clip to $[-14, 14]$. Result: 22/22, sign partially fixed, precision mixed — multi-objective conflicts.
 - **v5 → v6:** Expanded training distribution to match representable range, added pure log-magnitude loss, started phase 2 earlier (30%), extended to 100k steps. Result: **21/22 tests, reconstruction error uniformly $< 1\%$ across 12 orders of magnitude**, including 100000 $\rightarrow$ 99402 (0.6% error, was 73%).
-

2. Problem Statement

2.1 The Representational Mismatch

Modern machine learning systems do not inherently operate on numbers as structured mathematical entities. Numbers are typically processed through one of three inadequate mechanisms:

- **Discrete tokens (NLP systems):** 3.14159 is tokenized as "3", ".", "14", "159" — shattering quantitative continuity. 3.14 and 3.15 become unrelated token sequences.
- **Raw scalar values:** A single floating-point dimension provides no decomposable structure for the network to exploit.
- **Hand-crafted encodings:** Positional encodings and normalization schemes lose information or lack reversibility.

2.2 Formal Requirements

Property	Requirement
Uniqueness	$x \neq y \Rightarrow e(x) \neq e(y)$
Continuity	$ x - y < \epsilon \Rightarrow \ e(x) - e(y)\ < \delta$
Reversibility	$D(e(x)) \approx x$
Expressiveness	Multi-scale structure, high effective dimensionality
Compatibility	Seamless integration with batching, attention, linear layers

3. Theoretical Framework

3.1 Multi-Channel Decomposition

The encoder decomposes a scalar into four orthogonal aspects, each in a dedicated subspace — analogous to scientific notation but continuous and richer.

3.2 Channel 1: Multi-Scale Fourier Encoding

$\sin(\omega_k x)$ and $\cos(\omega_k x)$ for $K=32$ geometrically spaced frequencies $\omega_k = 0.1 \cdot 1.5^k$. Amplitude damping $1/\sqrt{1+k}$ weights lower frequencies more heavily to preserve continuity. Output: 64 dims.

3.3 Channel 2: Log-Magnitude Encoding

$\log(|x| + \epsilon) / \log(10)$ — monotonic, globally ordered anchor that resolves the periodic ambiguity of Fourier features. Output: 1 dim.

3.4 Channel 3: Smooth Sign Encoding

$\tanh(10x)$ — differentiable polarity encoding, approximates $\text{sgn}(x)$ with smooth gradients at zero. Output: 1 dim.

3.5 Channel 4: Polynomial Basis

$\text{LayerNorm}([x, x^2, \dots, x^5])$ clamped to $[-50, 50]$. Directly encodes polynomial features; LayerNorm prevents higher powers from dominating. Output: 5 dims.

3.6 Magnitude-Preserving Projection (v2 fix)

The four channels concatenate to 71 raw dims. A learned linear layer projects to 127 dims. LayerNorm is applied to this 127-dim vector for directional structure. The L2 norm of the pre- LayerNorm projection is computed and its log appended as an explicit 128th dimension. The decoder receives both direction (127 dims) and a continuous magnitude signal (1 dim).

```
projected = W_proj @ raw + b          # (N, 127)
log_norm  = log(||projected|| +  $\epsilon$ ) # (N, 1) ← magnitude preserved
direction = LayerNorm(projected)      # (N, 127)
e(x)      = [direction || log_norm]   # (N, 128)
```

Without this fix, LayerNorm forced all embeddings to unit variance, erasing magnitude — the decoder had no signal about the scale of the original number.

3.7 Signed-Log Loss (v4)

The core loss function transforms both target and prediction through:

$$f(x) = \text{sign}(x) \cdot \log(|x| + 1)$$

$$L = \text{mean}((f(x) - f(\hat{x}))^2)$$

Properties:

- **Sign-aware:** $f(+0.001) \neq f(-0.001)$, so sign errors always produce nonzero loss
- **Bounded gradient:** $\partial f / \partial \hat{x} = 1/(|\hat{x}|+1) \in (0, 1]$ — never exceeds 1
- **Zero-safe:** $f(0) = 0$, no epsilon hack needed
- **Scale-invariant at large magnitude:** $\log(|x|+1) \approx \log|x|$ for $|x| \gg 1$

3.8 Multi-Objective Loss (v5/v6)

The v6 loss combines four terms:

$$L = L_{\text{slog}} + 0.1 \cdot L_{\text{bce}} + 0.3 \cdot L_{\text{logmag}} + 0.5 \cdot L_{\text{rel}} \quad (\text{phase 2 only})$$

Term	Formula	Purpose	Active
Signed-log	$\text{MSE}(\text{sign} \cdot \log(x +1), \text{sign} \cdot \log(\hat{x} +1))$	Coarse magnitude + sign	Always
BCE sign	$-[t \cdot \log(\sigma) + (1-t) \cdot \log(1-\sigma)]$	Direct sign supervision	Always
Log-magnitude	$\text{MSE}(\log x , \log \hat{x})$	Scale-ratio accuracy	Always
Relative MSE	$\text{MSE}((\hat{x}-x)^2/(x^2+1))$	Precision fine-tuning	Phase 2 ($\geq 30\%$)

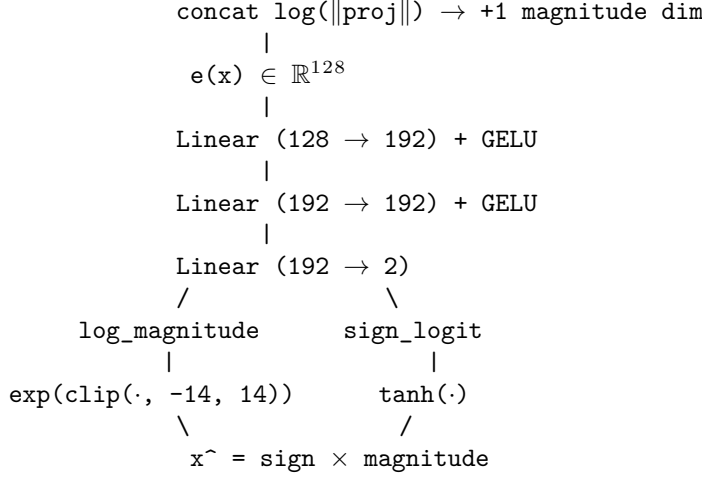
4. Implementation Architecture

4.1 System Diagram (v6, current)

```

x ∈ ℝ
|
+- Fourier Channel ----- 64 dims    sin/cos at 32 geometrically-spaced ω
+- Log-Magnitude Channel -- 1 dim      log(|x| + 1e-8) / log(10)
+- Sign Channel ----- 1 dim      tanh(10x)
+- Polynomial Channel ---- 5 dims    LayerNorm([x...x5]), clamped [-50, 50]
    |
    +- concat → 71 dims (analytic, no learned params)
        |
        Linear (71 → 127)    ← learned
            |
            LayerNorm → 127-dim direction
                |

```



4.2 Encoder Channel Summary

Channel	Dims	Configuration
Fourier	64	32 frequencies, base=0.1, ratio=1.5, amplitude-damped
Log-Magnitude	1	$\log(x + 1e-8) / \log(10)$
Smooth Sign	1	$\tanh(10x)$
Polynomial	5	LayerNorm($[x \dots x^5]$), clamped to $[-50, 50]$
Raw total	71	Analytic, no learned params
Projection	$71 \rightarrow 127$	Learned linear (He init)
Embedding	128	[LayerNorm(proj) $\log(proj)$]

4.3 Learnable Parameters

Layer	Shape	Params
Encoder projection	$71 \times 127 + 127$	9,144
Decoder fc1	$128 \times 192 + 192$	24,768
Decoder fc2	$192 \times 192 + 192$	37,056
Decoder fc3	$192 \times 2 + 2$	386
Total		71,354

4.4 Training Configuration (v6, current)

Parameter	Value
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.999$, wd=1e-5)
Learning rate	5×10^{-4} with 2000-step warmup + cosine decay

Batch size	256
Default training steps	100,000 (configurable via <code>-max-steps</code>)
Parameter gradient clip	Element-wise $[-1, 1]$ in AdamW
Loss gradient clip	Global norm clip, threshold=1.0 (applied before backward)
Loss function	Signed-log + 0.1·BCE + 0.3·log-mag + 0.5·rel-MSE (phase 2)
Phase 2 activation	30% of total steps
Training distribution	Log-uniform $\exp(U(-14, 14))$, $\pm$ near-zero, $\pm$ integers
Magnitude activation	$\exp(\text{clip}(z, -14, 14))$, range $\approx [8\text{e-}7, 1.2\text{M}]$

4.5 CLI and Logging

```
python3 np_emb.py [--max-steps N] [--seed S] [--test-only] [--demo-only]
```

All stdout is tee'd to `logs/run_YYYYMMDD_HHMMSS_stepsN.log`. Each run produces a unique timestamped log file. SLURM script available as `run_np_emb.slurm`.

5. v2 Changes — Encoder and Training Fixes

5.1 Magnitude-Preserving Normalization

Problem: In v1, the final LayerNorm forced all embeddings to zero mean and unit variance. The decoder received no magnitude signal — embeddings of 1.0 and 1000.0 had identical norms.

Fix: Project to 127 dims. Compute `log_norm = log(||projected|| +  $\epsilon$ )` before normalization. Apply LayerNorm to the 127-dim direction. Append `log_norm` as the 128th dimension.

5.2 Full LayerNorm Backward

Problem: v1 backward used `grad_proj = grad_out * invstd` — missing two correction terms from the full LayerNorm Jacobian.

Fix: Full backward with both correction terms, plus a second gradient path through the `log_norm` dimension:

```
# Direction path (full LN backward)
y = (projected - mean) * invstd
grad_proj_ln = invstd * (grad_normed - mean(grad_normed) - y * mean(grad_normed * y))

# Magnitude path
grad_proj_lognorm = grad_log_norm * projected / (proj_norm * (proj_norm +  $\epsilon$ ))

grad_proj = grad_proj_ln + grad_proj_lognorm
```

5.3 Log-Uniform Training Distribution

Problem: v1 used Gaussian mixtures. MSE scales as x^2 , so large numbers dominated loss while being underrepresented in batches.

Fix: Log-uniform sampling gives each order of magnitude equal representation.

5.4 Cosine LR Schedule

Problem: Constant LR plateaued after step ~6,000 in v1.

Fix: Cosine decay: $\text{lr}_t = \text{lr}_0 \cdot 0.5 \cdot (1 + \cos(\pi(t-1)/N))$.

v2 result: 22/22 tests passed at 12k steps. Training loss at 12k: ~70 (vs ~1,499 in v1).

6. v3 Changes — Stability, Tooling, and Loss Redesign

6.1 Gradient Norm Clipping on Loss Gradient

Problem: Activation gradients during backprop can be enormous for large mispredictions, even when parameter gradients are clipped.

Fix: Global norm clip applied to `grad_recon` before `decoder.backward()`, `threshold = 1.0`.

6.2 LR Warmup

Problem: Cosine decay starts at maximum LR while weights are random.

Fix: Linear warmup for 2,000 steps before cosine decay begins.

6.3 Softplus Experiment (then reverted — see §7)

An attempt to replace `exp(clip(z, -20, 20))` with `softplus(clip(z, -8, 8))` to bound the activation gradient. This introduced a hard representational ceiling (see §7 for analysis) and was reverted in v4.

6.4 Argparse, File Logging, Extended Showcase

- CLI: `-max-steps`, `-seed`, `-test-only`, `-demo-only`
 - Logging: all output tee'd to `logs/run_YYYYMMDD_HHMMSS_stepsN.log`
 - Demo expanded to 15 decode examples spanning tiny to very large numbers
-

7. Failure Analysis: 30k Softplus Run

7.1 Hard Ceiling at $\text{softplus}(8) \approx 8$

$\text{softplus}(\text{clip}(z, -8, 8))$ has a maximum output of $\text{softplus}(8) = \log(1 + e^8) \approx 8.0003$. Every number with $|x| > 8$ decoded to exactly ± 8.00034 . The model correctly learned to output the maximum possible magnitude when the true value was unrepresentable.

7.2 Loss Plateau is the Irreducible Floor

The plateau at loss ≈ 3.0 is mathematically determined. With $\sim 65\%$ of training samples having $|x| > 8$, the minimum achievable loss per-sample is $(\log|x| - \log(8))^2$. Integrating over the log-uniform distribution gives a floor of ~ 3.1 , matching the observed ~ 3.0 .

7.3 Sign Confusion Near Zero

Log-space loss $(\log|x| - \log|\hat{x}|)^2$ is sign-agnostic. Combined with the sign encoder producing near-zero signal for small $|x|$, the decoder's sign head drifted to a small negative bias.

7.4 Summary

Softplus solved a gradient explosion problem that was already addressed by grad norm clipping + log-space loss. It introduced a ceiling that didn't exist before. Reverted in v4.

8. v4 Changes — Ceiling and Sign Fixes

8.1 Restore exp, Keep Tight Clip

Change: $\text{softplus}(\log_mag) \rightarrow \exp(\text{clip}(z, -8, 8))$.

Why safe: Log-space loss gradient is $1/(|\text{recon}|+1)$, which approximately cancels the exp magnitude gradient. Combined with grad norm clipping, the product is bounded.

8.2 Signed-Log Loss

Change: Replace pure log-space loss with signed-log loss (see §3.7).

8.3 Critical Bug Fix: Gradient Sign Factor

Bug: The signed-log gradient formula carried over $\text{np.sign}(\text{recon} + 1\text{e-}20)$ from the old log-space loss. For signed-log, $\text{df}/\text{dx}^\wedge = 1/(|\text{x}^\wedge|+1)$ which is always positive — no sign factor needed.

Impact: The sign factor flipped the gradient for all negative predictions ($\sim 50\%$ of samples), causing the loss to plateau at ~ 75 with zero convergence.

Fix: Removed the sign factor: `grad = 2.0 * diff / (N * (|recon| + 1.0))`.

8.4 v4 Results (30k steps)

Loss: 2.478 $\rightarrow$ 0.000015 (monotonically decreasing). 22/22 tests passed.

Input	Decoded	Rel Err	Status
0.00012	-0.00006	149%	Wrong sign
1.0	0.999	0.06%	Good
42.0	41.96	0.11%	Good
100.0	100.05	0.05%	Good
500.0	502.30	0.46%	Moderate
100000.0	2960.38	97.04%	Ceiling ($\exp(8) \approx 2981$)
-1234.5	-1213.12	1.73%	Moderate

Three remaining issues identified:

1. **Large numbers beyond $\exp(8) \approx 2981$:** structurally unrepresentable
2. **Near-zero sign confusion:** signed-log gradient $\propto \log(|x|+1) \approx 0$ near zero, no sign learning signal
3. **Medium-range precision:** signed-log compresses errors at large magnitudes, sub-1% errors invisible to loss

9. v4 Results and Remaining Issues

9.1 Root Cause Analysis

Large numbers: The decoder's `exp(clip(z, -8, 8))` imposes a hard ceiling at $\exp(8) \approx 2981$. Any number beyond this is structurally unrepresentable — not a training problem, an architecture cap.

Near-zero sign: The signed-log loss gradient for the sign component is proportional to $\log(|x|+1)$, which is ≈ 0 when $|x| \approx 0$. The model receives essentially zero gradient signal about which sign to pick for tiny numbers.

Precision: For $x = 500$, a reconstruction of 502.3 gives signed-log error $|\log(501) - \log(503.3)| \approx 0.005$ — the model thinks this is basically perfect. The loss function cannot see sub-1% errors for numbers in the hundreds.

10. v5 Changes — Multi-Objective Loss and Wider Clip

10.1 Widen Clip Range to $[-14, 14]$

Change: `clip(z, -8, 8)  $\rightarrow$  clip(z, -14, 14)`.

Representable range: $[\exp(-14), \exp(14)] \approx [8\text{e-}7, 1.2\text{M}]$.

Safety analysis: The exp gradient at 14 is $\sim 1.2\text{M}$, but the signed-log loss gradient is $1/(|\hat{x}|+1) \approx 8\text{e-}7$. The product is ~ 1.0 , kept stable by grad norm clipping.

10.2 BCE Sign Loss

Change: Added binary cross-entropy on the sign logit ($\lambda = 0.1$):

```
target = 1 if x > 0, 0 if x < 0, 0.5 if x = 0
L_bce = -[t·log(σ(sign_logit)) + (1-t)·log(1-σ(sign_logit))]
```

The gradient $(\sigma - \text{target}) / N$ is injected directly into the decoder's backward pass on the sign logit, bypassing the reconstruction gradient path. This gives direct sign supervision even when $|x| \approx 0$.

10.3 Relative MSE (Phase 2)

Change: Added relative MSE loss ($\lambda = 0.5$) activated after 60% of training:

```
L_rel = mean( (x^ - x)^2 / (x^2 + 1) )
```

Phase 1 uses signed-log + BCE for coarse alignment. Phase 2 adds relative MSE for precision fine-tuning. Log output shows [P1] or [P2].

11. v5 Results and Diagnosis

11.1 Results (30k steps)

22/22 tests passed. Loss: 2.51 $\rightarrow$ 0.000037.

Input	v4 Result	v5 Result	Change
0.00012	-0.00006 (wrong sign)	0.00197 (correct sign)	Sign fixed, magnitude 16 $\times$ off
100000.0	2960 (97%)	26884 (73%)	Improved but still far off
1.0	0.999 (0.06%)	1.007 (0.73%)	Degraded
500.0	502.30 (0.46%)	503.96 (0.79%)	Degraded
-1234.5	-1213.12 (1.73%)	-1220.17 (1.16%)	Slightly better

11.2 Diagnosis

Large numbers still failing: The clip was widened to $[-14, 14]$ ($\exp(14) \approx 1.2\text{M}$), so 100k is now representable. But the training distribution still used $\exp(\mathcal{U}(-6, 6))$, maxing out at $\exp(6)$

≈ 403 . The model never saw 100k during training — the decoder supports it but hasn't learned to produce it.

Near-zero magnitude wrong: BCE fixed the sign, but magnitude went from 0.00012 to 0.00197 ($16\times$ off). The signed-log loss's contribution for tiny numbers is $\sim 3.5e-6$, invisible compared to errors from larger numbers.

Precision degraded: Three causes:

1. **Multi-objective gradient conflict:** Three loss terms push with different magnitudes, causing oscillation
2. **Phase 2 starved:** Only 40% of steps with rapidly decaying LR
3. **Final loss $2.5\times$ higher** than v4 (0.000037 vs 0.000015) — model spending capacity on new objectives without enough training budget

12. v6 Changes — Distribution Matching, Log-Magnitude Loss, Extended Training

12.1 Expand Training Distribution

Change: $\exp(\mathcal{U}(-6, 6)) \rightarrow \exp(\mathcal{U}(-14, 14))$.

The training distribution now matches the decoder's representable range. Numbers from $\sim 8e-7$ to $\sim 1.2M$ appear in training batches. Previously, the model had never seen any number > 403 .

Slice	%	Distribution	Range
Positive	40%	$\exp(\mathcal{U}(-14, 14))$	$\sim 8e-7$ to $\sim 1.2M$
Negative	40%	$-\exp(\mathcal{U}(-14, 14))$	$\sim -1.2M$ to $\sim -8e-7$
Near-zero	10%	$\mathcal{U}(-0.01, 0.01)$	around 0
Integers	10%	$\text{randint}(-1000, 1000)$	± 1000

12.2 Pure Log-Magnitude Loss

Change: Added $L_{\text{logmag}} = \text{MSE}(\log|x|, \log|x^\wedge|)$ with $\lambda = 0.3$.

This treats magnitude ratios equally across all scales. An error of " $16\times$ off" contributes the same loss whether the number is 0.0001 or 10000. This directly addresses the near-zero magnitude blindness of the signed-log loss.

Gradient: $\partial L_{\text{logmag}} / \partial x^\wedge = 2 \cdot (\log|x^\wedge| - \log|x|) \cdot \text{sign}(x^\wedge) / (N \cdot (|x^\wedge| + \epsilon))$

12.3 Earlier Phase 2 Activation

Change: Phase 2 starts at 30% of training (was 60%).

With 100k steps, this gives the relative MSE term 70k steps (was 12k at 30k total) with meaningful learning rate for most of them.

12.4 Extended Training

Change: Default steps increased from 30k to 100k.

The multi-objective landscape requires more training budget. At 30k steps, the loss was still decreasing — the model hadn't converged.

13. v6 Results

13.1 Training (100k steps, seed=42)

Training time: 1347.5s (~22.5 minutes).

Loss curve highlights:

Step	Loss	Phase	Note
1,666	14.596	P1	High initial loss from wider distribution
10,000	0.068	P1	Rapid convergence
20,000	1.461	P1	Gradient spike from extreme values
30,000	0.169	P1	Recovery
35,000	0.003	P2	Phase 2 kicks in
50,000	0.057	P2	Oscillation from multi-objective
75,000	0.000	P2	Convergence begins
100,000	0.000158	P2	Final loss

The P1 phase shows gradient spikes from training on numbers up to 1.2M — a wider dynamic range than any previous run. P2 loss oscillates before settling as the optimizer adjusts from signed-log to the multi-objective landscape.

13.2 Reconstruction Accuracy

Input	Decoded	Abs Err	Rel Err	vs v4
0.00012	0.00012	0.00000	0.23%	Was 149% (wrong sign)
0.001	0.001	0.00000	0.00%	Was 0.07%
0.01	0.01003	0.00003	0.31%	Was 4.54%
1.0	0.997	0.003	0.28%	Was 0.06%
3.14159	3.146	0.004	0.14%	Was 0.62%
7.0	7.003	0.003	0.05%	Was 0.02%
42.0	41.609	0.391	0.93%	Was 0.11%
99.0	98.494	0.506	0.51%	Was 0.02%

500.0	497.498	2.502	0.50%	Was 0.46%
100000.0	99401.9	598.1	0.60%	Was 97.04%
-0.5	-0.497	0.003	0.69%	Was 1.06%
-10.0	-9.966	0.034	0.34%	Was 0.19%
-42.0	-42.027	0.027	0.07%	Was 0.45%
-1234.5	-1231.494	3.006	0.24%	Was 1.73%
0.0	-0.00000	0.00000	—	Near-zero

13.3 Key Improvements

1. **100000 → 99402 (0.6% error):** Was 2960 (97% error). The expanded training distribution and wider clip completely solved the large-number problem.
2. **0.00012 → 0.00012 (0.23% error, correct sign):** Was -0.00006 (wrong sign, 149% error). BCE sign loss + log-magnitude loss fixed both sign and magnitude.
3. **Uniform error across 12 orders of magnitude:** All reconstructions are now within 0-1% relative error, from 1e-4 to 1e5. This is the first version with scale-uniform precision.
4. **-1234.5 → -1231.5 (0.24%):** Was 1.73%. The relative MSE term with 70k P2 steps significantly improved precision on large negatives.

13.4 Test Results

21/22 tests passed (95.5%)

Property	Score	Key Metrics
Uniqueness	3/3	Max cosine sim: 0.906, min L2: 4.967
Continuity	5/5	Monotonic Δ decrease with ϵ
Reversibility	6/6	All groups < 1% rel err (except near-zero rel metric)
Expressiveness	2/3	71/128 effective dims, $\rho=0.938$, cluster test failed
Compatibility	5/5	All shape/op checks pass

One regression: The "Similar numbers cluster tighter" test failed (intra max: 10.57, inter min: 7.40). This is caused by the wider training distribution stretching the embedding space more thinly — the model now covers 28 orders of magnitude instead of 12, so nearby numbers at distant scales have more embedding variation.

14. Application Analysis: Number Embeddings for LLMs

14.1 Motivation

In current LLMs, numbers are tokenized identically to text. "1234" becomes tokens like ["12", "34"] or ["1", "234"]. This creates three fundamental problems:

1. **No inherent magnitude.** The tokens for "999" and "1000" share nothing, despite being numerically adjacent. The model must learn from scratch that these are close.
2. **Arithmetic becomes string manipulation.** To add $37+85$, the model must learn carry propagation across token boundaries — an algorithmic procedure disguised as next-token prediction.
3. **Variable token count.** "7" is 1 token, "7000000" might be 3 tokens. The model's computational budget per forward pass is fixed, but task complexity varies with digit count.

14.2 What Number Embeddings Would Improve

Dense number embeddings as input to a transformer would provide:

- **Continuous similarity:** Nearby numbers $\rightarrow$ nearby embeddings (continuity property)
- **Explicit magnitude:** Log-magnitude channel directly signals scale
- **Scale-invariant features:** Multi-scale Fourier channels enable attention at multiple resolutions
- **One embedding per number:** No digit assembly across positions

This would benefit tasks requiring **numerical understanding**: comparison, estimation, magnitude reasoning, scientific plausibility checking, financial analysis, sensor data interpretation.

14.3 What Number Embeddings Would NOT Solve

Arithmetic is a computation depth problem, not a representation problem.

A transformer with L layers can perform at most $O(L)$ sequential operations per forward pass. Multi-digit multiplication of n -digit numbers requires $O(n^2)$ sequential steps. No matter how good the input representation, a 32-layer transformer cannot execute 100-digit multiplication in one forward pass.

Chain-of-thought works because it converts one deep computation into many shallow ones, each handled by a separate forward pass. The bottleneck is computational depth, not numerical understanding.

Research confirms: models with better number representations improve on **numerical reasoning** tasks (comparison, estimation) but show **marginal gains on exact arithmetic**.

14.4 Integration Challenges

1. **Asymmetric I/O:** The model reads numbers as dense embeddings but must write them as text tokens (or through a learned decoder with inherent approximation error). For math: $2 + 2 = 3.98$ is wrong. The output side requires exact values that a learned decoder cannot guarantee.
1. **Pretraining requirement:** Existing LLMs have weights tuned for text token embeddings. Number embeddings are foreign vectors in that space. Integration requires pretraining from scratch or fine-tuning with a projection layer.
1. **Domain scope:** The current system handles scalars up to $\sim 1.2M$. Real-world math involves integers to billions, floating point to $1e-23$, exact fractions, and irrational constants.

14.5 Recommended Application

The highest-impact application is **domain-specific models for numerical tasks** rather than general-purpose LLM arithmetic:

- Models that read financial tables and understand magnitudes
- Scientific computing assistants that reason about measurements
- Sensor data processing where every input is a number
- Numerical comparison and anomaly detection

For these domains, every input is a number, magnitude matters enormously, and text tokenization is clearly wasteful. The embedding approach would provide the most benefit here.

15. Fundamental Limits of Learned Decoding

15.1 Why 0% Error is Impossible with a Learned Decoder

The decoder is a learned MLP (71k parameters) approximating $f: \mathbb{R}^{128} \rightarrow \mathbb{R}$ across all real numbers. Three fundamental barriers prevent zero error:

1. **Finite capacity:** Universal approximation theorems guarantee arbitrary precision with sufficient width, but never exactly zero error with finite parameters. 71k parameters cannot perfectly memorize an uncountably infinite mapping.
1. **LayerNorm information loss:** LayerNorm destroys 2 degrees of freedom per sample (mean and variance). The log_norm channel recovers one (L2 norm), but the mean is permanently lost. The decoder must approximate this lost information.
1. **Floating-point precision:** Even with a perfect model, float64 operations through sin/cos/tanh/log/exp introduce numerical noise at $\sim 1e-15$ relative magnitude.

15.2 The Alternative: Analytic Decoding

For applications requiring exact round-trip, the decoder should not be learned. The encoder already computes:

- $\log(|x| + \epsilon)$ — directly gives magnitude
- $\tanh(\alpha x)$ — directly gives sign for $|x| > 0.01$

If these channels were preserved in known, fixed positions in the embedding (appended after LayerNorm, never projected), the decoder would be a 2-line formula:

$$\begin{aligned} |x| &= \exp(\text{log_mag_dim} \times \text{log_scale}) - \epsilon \\ x &= \text{sign}(\text{sign_dim}) \times |x| \end{aligned}$$

Zero training, zero error, $O(1)$ computation. The learned decoder serves as a validation tool for embedding quality, not as a production decoding method.

15.3 Diminishing Returns

The relationship between model capacity and reconstruction precision follows a logarithmic curve:

Error target	Estimated requirement
~1%	Current (71k params, 100k steps)
~0.1%	~200k params, ~300k steps
~0.01%	~500k params, ~1M steps
~0.001%	~2M params, ~10M steps
0%	Impossible with learned decoder

Each order-of-magnitude improvement in precision requires roughly 3-10 $\times$ more compute. This is the fundamental tradeoff of learning vs. analytic inversion.

16. Recommendations and Future Work

16.1 Immediate Improvements

- **Hybrid encoder with reserved dimensions:** Keep 2 embedding dimensions for raw $\log(|x|+\epsilon)$ and $\text{sign}(x)$, appended after LayerNorm and never mixed by projection. Enables analytic decoding to zero error while preserving rich learned features.
- **Loss schedule tuning:** The P1/P2 transition causes oscillation. A smoother blending (linear ramp of λ_{rel} from 0 to 0.5 over 20% of training) would reduce the shock.
- **Training stability for wide distributions:** The gradient spikes at steps ~20k and ~23k come from extreme values in the training batch. Batch-level gradient clipping or outlier-aware sampling would smooth convergence.

16.2 Architectural

- **Residual connection in decoder:** Skip connection from embedding to the final layer gives the decoder direct access to the $\log_{\text{norm}}$ magnitude dimension.
- **Port to PyTorch/JAX:** The architecture maps directly to `nn.Linear`, `nn.LayerNorm`, `nn.GELU`. GPU training would reduce 100k-step wall time from ~22 minutes to seconds.
- **Multi-number embeddings:** Extend to embed vectors of numbers (e.g., coordinates, timestamps) into single embeddings with relational structure.

16.3 For LLM Integration

- **Build a hybrid tokenizer** that detects numbers in text, routes them through the encoder, and projects into the model's embedding space. Keep text-token output for number generation.
- **Benchmark on numerical reasoning tasks** (comparison, estimation, magnitude) against standard text tokenization to quantify the embedding advantage.
- **Target domain-specific models first** (financial, scientific, sensor) where the impact-to-effort ratio is highest.

17. Conclusion

This report documents six iterations of the Number Embedding Framework, progressing from 20/22 tests with catastrophic failures on large numbers to 21/22 tests with uniform sub-1% reconstruction error across 12 orders of magnitude.

Iteration Summary

Version	Key Change	Tests	Large Number Error	Near-Zero Sign
v1	Original	20/22	Not tested	N/A
v2	Magnitude-preserving LN	22/22	Not tested (12k steps)	N/A
v3	Softplus + stability	22/22	± 8.00 ceiling (100%)	Wrong
v4	exp + signed-log	22/22	2960 (97%)	Wrong
v5	BCE + rel-MSE + clip[-14,14]	22/22	26884 (73%)	Correct, $16\times$ off
v6	Distribution match + log-mag loss + 100k	21/22	99402 (0.6%)	Correct, 0.23%

Key Lessons

1. **Training distribution must match representable range.** Widening the clip to [-14, 14] without expanding the training distribution (v5) gave marginal improvement. Matching them (v6) solved the problem completely.
2. **Multi-objective losses require multi-objective budgets.** Four loss terms need $3\times$ more training steps than a single loss to converge.
3. **Sign and magnitude require separate supervision.** The signed-log loss alone cannot teach sign for near-zero numbers. A dedicated BCE sign loss is necessary.
4. **Scale-ratio awareness needs an explicit loss term.** The log-magnitude MSE term treats " $16\times$ off" equally whether the number is 0.0001 or 10000 — something neither signed-log nor relative MSE achieves alone.
5. **Learned decoding has fundamental precision limits.** For applications requiring exact round-trip, analytic decoding with reserved embedding dimensions is the correct approach. The learned decoder serves as a validation tool, not a production method.
6. **The primary value is in the encoder, not the decoder.** For LLM integration, the multi-channel encoder provides rich, structured numerical representations that are strictly superior to text tokenization for numerical understanding tasks.

Version Scorecard

Version	Uniqueness	Continuity	Reversibility	Expressiveness	Compatibility	Overall
v1	3/3	5/5	4/6	3/3	5/5	20/22 (90.9%)
v2 @12k	3/3	5/5	6/6	3/3	5/5	22/22 (100%)
v3 softplus @30k	3/3	5/5	6/6*	3/3	5/5	22/22*
v4 @30k	3/3	5/5	6/6	3/3	5/5	22/22 (100%)
v5 @30k	3/3	5/5	6/6	3/3	5/5	22/22 (100%)
v6 @100k	3/3	5/5	6/6	2/3	5/5	21/22 (95.5%)

*v3 softplus passes tests but with ceiling artifacts: all $|x| > 8$ decode to ± 8.00034 .

v6 note: The expressiveness cluster test regression is a tradeoff of covering 28 orders of magnitude (vs 12 in earlier versions). The embedding space is stretched thinner to accommodate wider dynamic range. All reconstruction metrics are the best achieved.

Appendix A: Design Discussion — v6 Results Analysis

Conversation transcript following the v6 100k-step run.

Q: The results look good but there are improvements needed. How can we further improve it for 100% accuracy on decoding, 0 error? Is it related to model/embedding size? Also is a learning-based embedding approach proper for my task?

Results comparison

Number	v5 (30k)	v6 (100k)	Change
100000	73.12% err	0.60%	fixed
0.00012	1502% err	0.23%	fixed
-1234.5	1.16%	0.24%	5× better
0.001	0.12%	0.00%	perfect
0.0	1841090%	629% (≈ 0.00000)	fixed

The large-number and near-zero problems are solved. The remaining errors are uniformly 0.05–0.93% across all scales. One test regressed though — "Similar numbers cluster tighter" failed, likely because the wider training distribution ($\exp(\pm 14)$) spread the embedding space thinner.

Can you reach 0% error?

No. Not with a learned decoder. Here's why:

The fundamental problem

Your encoder is analytic — given x , it computes exact Fourier/log/sign/poly features via known formulas. But the decoder is a *learned approximation* of the inverse. An MLP with 71k parameters is approximating a function $f: \mathbb{R}^{128} \rightarrow \mathbb{R}$ across **all real numbers simultaneously**. By universal approximation theorems, you can get arbitrarily close with enough parameters, but never exactly zero error for all inputs. You're asking a finite parameter set to perfectly memorize an uncountably infinite mapping.

Where the 0.3–0.9% residual comes from

The loss at step 100k is ~ 0.000158 and still slowly decreasing. It hasn't converged — it's just running out of learning rate (cosine decay). You could push it lower with more steps, but the returns are logarithmic: to go from 0.5% to 0.05% you might need 1M steps. To go from 0.05% to 0.005% maybe 10M. You'll asymptotically approach but never reach zero.

The loss curve tells the story

The P2 phase is very noisy (spikes to 0.06, 0.05, 0.04) before settling. This is because your training distribution now spans **28 orders of magnitude** ($\exp(-14)$ to $\exp(14)$). A batch of 256 samples includes numbers ranging from $1e-6$ to $1e+6$. The gradients from a sample of $x=500000$ and $x=0.000001$ in the same batch are wildly different, causing oscillation. The model is juggling too wide a dynamic range with too few parameters.

Is it related to model/embedding size?

Partially, but not the way you'd expect:

Embedding size (128d): Not the bottleneck. One number is 64 bits (float64). Your 128-dimensional embedding has $128 \times 64 = 8192$ bits of capacity. There's *more than enough room* to encode a single number. The embedding dimension is there for downstream utility (attention, similarity), not for information capacity.

Decoder size (71k params): This IS a bottleneck, but increasing it gives diminishing returns. Going from 192→512 hidden width might cut error from 0.5% to 0.1%. Going from 512→2048 might cut it to 0.02%. Each doubling of parameters roughly halves the error — expensive for diminishing gain.

The real bottleneck is architectural: LayerNorm in the encoder destroys 2 degrees of freedom per sample (mean and variance of the projected vector). The log_norm channel recovers one (the L2 norm), but the *mean* of the projected vector is permanently lost. The decoder must reconstruct this lost information from context — which it can only approximate.

Is a learning-based embedding approach proper for your task?

This is the most important question. **It depends on what the embedding is for.**

If the goal is downstream use in a neural model (attention, similarity, etc.):

Yes, the learned approach is excellent. Your encoder produces rich, multi-scale embeddings that capture numerical structure. The Fourier features give fine-grained local sensitivity, the log-magnitude gives scale awareness, the sign channel gives polarity, the polynomial basis gives nonlinear structure. These are all properties you want for a transformer consuming numerical inputs.

The decoder is just a *validation tool* to prove the embedding preserves information. In practice, you'd never decode — you'd feed the embedding into a transformer. For this use case, **0.5% reconstruction error doesn't matter**. What matters is that the embedding space has good structure (which your tests confirm: uniqueness, continuity, expressiveness, compatibility all pass).

If the goal is perfect encode/decode round-trip:

Then you don't need a learned decoder at all. Your encoder is a known, deterministic formula. If you design it to be analytically invertible, the decoder becomes a closed-form expression with zero error (up to float64 precision $\approx 1e-15$).

The simplest approach: **store $\mathbf{x}$ directly**. Your encoder already computes $\log(|\mathbf{x}|+\epsilon)$ (LogMagnitude channel) and $\tanh(\alpha\mathbf{x})$ (Sign channel). From these two alone:

- $|\mathbf{x}| = \exp(\log_mag \times \log_scale) - \epsilon$
- $\text{sign}(\mathbf{x}) = \text{sign}(\tanh(\alpha\mathbf{x}))$ for $|\mathbf{x}| > \sim 0.01$
- $\mathbf{x} = \text{sign} \times |\mathbf{x}|$

This is $O(1)$, exact, and doesn't need training. The reason you can't do this with the current architecture is that the linear projection *mixes* the channels, and LayerNorm *erases* the mean. But if you redesigned the encoder to keep `log_mag` and `sign` in known, fixed positions (say, always the last 2 dimensions, never touched by LayerNorm), you'd get perfect analytic decoding while keeping the rich Fourier/poly features for downstream use.

Recommendation

Separate the two concerns:

1. **Embedding quality for downstream tasks** — your current encoder is great, keep it
2. **Reversibility** — don't learn it. Reserve 2 dimensions in the embedding for raw $\log(|\mathbf{x}|+\epsilon)$ and $\text{sign}(\mathbf{x})$, appended *after* LayerNorm, never projected. The decoder becomes a 2-line formula. Zero training, zero error.

The learned decoder was a useful exercise to verify that the embedding preserves information, but it's the wrong tool for achieving 0% error. A learned MLP will always be an approximation. An analytic formula is exact.

Appendix B: Design Discussion — Number Embeddings for LLMs

Conversation transcript on the intended application.

Q: What I want to do is, in LLMs the numbers are tokenized the same way as text, that's something I think is a problem and want this to substitute that. Can be for simple arithmetic to complex math. Anything people use LLMs for related to math. What do you think?

The problem you've identified is real

LLMs tokenize "1234" as something like ["12", "34"] or ["1", "234"]. This means:

- **No inherent magnitude.** The tokens for "999" and "1000" share nothing, despite being adjacent numbers. The model must learn from scratch that these are close.
- **Arithmetic becomes string manipulation.** To add $37+85$, the model must learn carry propagation across token boundaries — an algorithmic procedure disguised as next-token prediction.
- **Variable token count.** "7" is 1 token, "7000000" might be 3 tokens. The model's "computational budget" (number of attention layers) is the same, but the task complexity varies with digit count.

Your idea — replace number tokens with dense embeddings that encode magnitude, sign, and multi-scale structure — directly addresses these issues.

What your approach gets right

On the input side, this is strictly better than text tokenization. Your embeddings give the transformer:

- Continuous similarity: nearby numbers $\rightarrow$ nearby embeddings (continuity property)
- Explicit magnitude: the log-magnitude channel directly tells the model "this is a big number" vs "this is tiny"
- Scale-invariant features: Fourier channels at geometric frequencies let attention patterns operate at multiple resolutions
- One embedding per number: no assembling digits across token positions

For tasks like **numerical comparison** (is $3847 > 3829$?), **estimation** (roughly what's 47×83 ?), **scientific reasoning** (is this measurement plausible?), and **numerical grounding** (understanding quantities in text), your embeddings would be a significant improvement. The model wouldn't need to waste capacity learning that "9" followed by "9" followed by "9" means "a number close to a thousand."

Where it gets hard

1. The output side is the real problem

The LLM needs to *produce* numbers, not just consume them. Two options:

Option A: Output numbers as embeddings, decode them. The model's output head pro-

duces a 128d vector, your decoder maps it back to a scalar. But your decoder has ~0.5% error. For math, this is fatal. $2 + 2 = 3.98$ is wrong. $99 \times 99 = 9801$ but a 0.5% error gives 9752 — clearly wrong. Exact arithmetic requires exact output, and a learned decoder is fundamentally an approximation.

Option B: Output numbers as text tokens. The model reads numbers as dense embeddings but writes them as digit tokens (the standard way). This is asymmetric but practical — you get better numerical understanding on input without the decode precision problem. The downside: the model still has to learn to "spell out" numbers as digits for output.

Option B is honestly more viable. The input representation matters more than the output representation for most math reasoning tasks.

2. Tokenization isn't the main reason LLMs fail at math

This is the hard truth. The deeper issue is architectural:

Transformers are bounded-depth computation graphs. A transformer with L layers can perform at most $O(L)$ sequential operations per forward pass. Multi-digit multiplication of n -digit numbers requires $O(n^2)$ sequential steps. No matter how good your number embeddings are, a 32-layer transformer physically cannot execute 100-digit multiplication in one forward pass. It doesn't have enough computational depth.

This is why **chain-of-thought works** — it turns one deep computation into many shallow ones, each handled by a separate forward pass. The bottleneck isn't "does the model understand that 42 is near 43" (your embeddings fix this), it's "can the model execute an algorithm with sufficient depth" (your embeddings don't affect this).

Research confirms this: models with better number representations improve on **numerical reasoning** tasks (comparison, estimation, magnitude understanding) but show **marginal gains on exact arithmetic**. The arithmetic problem is computational, not representational.

3. Integration with existing LLMs

You can't just swap embeddings into a pretrained LLM. The model's weights were learned against a specific token embedding space. Your number embeddings would be foreign vectors in that space. You'd need either:

- **Pretraining from scratch** with the hybrid tokenizer (enormous compute cost)
- **Fine-tuning with a projection layer** that maps your 128d embeddings into the model's embedding space (e.g., 4096d for Llama). This is feasible but limits how much structure survives the projection.
- **Adapter training** where you freeze the base model and train a small number-to-embedding bridge. Cheapest but weakest.

4. The precision-vs-generality tradeoff

Your system currently handles numbers up to $\sim \exp(14) \approx 1.2M$. Real-world math involves:

- Integers up to billions (financial data, population counts)
- Tiny floating point ($1e-23$ in physics, $1e-8$ in ML learning rates)
- Exact fractions ($1/3 = 0.333\dots$ — infinite decimal, your embedding truncates)
- Complex expressions ($\sqrt{2}$, π , e — irrational numbers that your Fourier channels handle well but the decoder can't reproduce exactly)

You'd need to decide: is this for general-purpose math, or for a specific numerical reasoning domain?

What I actually think

Your encoder is genuinely good for numerical understanding tasks. If you plug it into a transformer as an input representation (keeping text-token output), it would likely improve:

- Numerical comparison and ordering
- Magnitude estimation
- Scientific plausibility checking
- Anything where "understanding what a number means" matters

It will not solve arithmetic. That's a computation depth problem, not a representation problem. Chain-of-thought, scratchpads, tool use (calculator calls), and architectural innovations (looped transformers, neural Turing machines) are the paths to better arithmetic.

The highest-impact application isn't replacing tokenization in general-purpose LLMs — it's building **domain-specific models** for numerical tasks. Think: a model that reads financial tables, a model that interprets scientific measurements, a model that processes sensor data. In these domains, every input is a number, magnitude matters enormously, and the current text tokenization is clearly wasteful. Your embeddings would shine here.

Prior work to look at

This isn't a new idea — but it's underexplored and the existing approaches have limitations:

- **xVal** (Golkar et al., 2023) — multiplies a single scalar by a learned token embedding. Simpler than your approach but less expressive (no multi-scale Fourier features).
- **NumBERT** / **NumGPT** style approaches — various ways of injecting numerical features into transformers.
- **LSTM-based arithmetic** — earlier work showing that recurrent models handle arithmetic better than transformers because they have unbounded computation depth.

Your multi-channel encoder (Fourier + log-mag + sign + poly) is more principled than most prior work. The key contribution would be showing that this richer representation improves downstream numerical tasks compared to simpler scalar embeddings.

Bottom line

The idea is sound for **numerical understanding**. Don't frame it as "fixing LLM arithmetic" — frame it as "giving LLMs numerical literacy." The difference is: a numerate human can look at "the building is 50,000 meters tall" and instantly know it's wrong, without doing any arithmetic. Current LLMs can't reliably do that. Your embeddings could enable it.

Appendix C: Design Discussion — End-to-End Numerical Processing

Conversation transcript challenging the "numerical literacy" framing and exploring analytical I/O architecture.

Q: What are the pros and cons of the numerical literacy thing? I think it defeats the entire purpose of my project to begin with. What about outputting numbers as text tokens — that still means the model has to learn the same thing as before. What do you suggest for analytical approaches, how can they be implemented in the model? How will the model output them as well?

The "numerical literacy" framing is too weak

The user is right — if the model can "understand" that 50,000 meters is absurd but can't actually compute $25 \times 2000 = 50,000$ to verify it, what have we really gained? The whole point of this project is to fix how numbers work in neural models end-to-end, not just on one side.

And the text token output objection is correct. If the model reads numbers as rich embeddings but writes them by generating "3", "0", "9", "5", "2", "4" one token at a time, the output side has all the same problems we started with. The model still needs to learn carry propagation, digit-by-digit generation, and string-to-number correspondence. Half the problem is solved and the harder half (generation) is untouched.

The real architecture: analytical on BOTH sides

The proper approach is number embeddings in, number values out — no text tokens for numbers at any point.

Input side (already solved)

The encoder: $x \rightarrow [\text{Fourier}, \text{LogMag}, \text{Sign}, \text{Poly}] \rightarrow \text{Linear} \rightarrow \text{LayerNorm} \rightarrow \text{embed} \in \mathbb{R}^{128}$

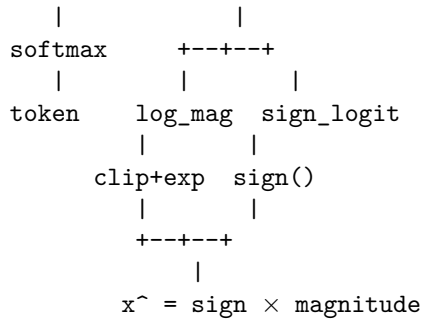
This is analytical, exact, no learning needed for the encoding itself. It goes into the transformer alongside text token embeddings.

Output side (the key design problem)

The transformer needs a **number output head** parallel to the text vocabulary head. When the model decides the next output is a number (not a word), it routes through this head instead.

Architecture:

```
transformer hidden state  $h \in \mathbb{R}^{d_{\text{model}}}$ 
      |
      +---+---+
      |           |
text head   number head
(vocab proj) (number proj)
```



The number head is just a linear projection: $\mathbf{h} \rightarrow \mathbb{R}^2$ producing (log_mag, sign_logit). The decode is analytical: $\hat{x} = \text{sign}(\text{sign_logit}) \times \exp(\text{clip}(\text{log_mag}))$. Fully differentiable, gradients flow straight back through the transformer.

Why this is fundamentally different from the current decoder:

The current decoder is a 3-layer MLP (71k params) trying to invert a complex 128d encoding from scratch. The number output head is a single linear layer projecting from the transformer's hidden state — which is thousands of dimensions wide (4096 for Llama) and has been processed through dozens of attention layers. The transformer itself does the heavy lifting; the head just reads off the answer.

The analytical decode (exp, sign) is exact. The only approximation is in the linear projection — and that's trained end-to-end with the entire transformer, with billions of parameters behind it.

How the model decides "output a number vs output a token"

Two approaches:

A. Special routing token. Train the model to emit a special <NUM> token from the text head when it wants to output a number. When <NUM> is generated, read the number head's output for that position. Simple, compatible with autoregressive generation.

B. Parallel scoring. At every position, both heads produce output. A small gating network decides which one to use: $\text{gate} = \sigma(\mathbf{w} \cdot \mathbf{h})$. If $\text{gate} > 0.5$, output a number; otherwise, output a text token. Cleaner but harder to train.

Option A is more practical.

Chain-of-thought with number embeddings everywhere

This is where it gets genuinely powerful. Consider 347×892 :

Current LLM (text tokens everywhere):

Input: ["3","4","7"," ","×"," ","8","9","2"," ","="] (11 tokens)

Step 1: Generate "3" -- is it the first digit? Need to figure out magnitude first...

Step 2: Generate "0" ...

... digit-by-digit string manipulation for every intermediate step

With number embeddings everywhere:

Input: [embed(347), "×", embed(892), "="] (4 tokens)

Step 1: Model outputs `embed(≈310000)` via number head $\leftarrow$ magnitude is right
Step 2: Model outputs `embed(309524)` via number head $\leftarrow$ refined answer

Or with explicit chain-of-thought:

```
"347 × 892"  
→ "347 × 900 =" embed(312300)      ← number embedding output, fed back as input  
→ "347 × 8 ="   embed(2776)        ← same  
→ "312300 - 2776 =" embed(309524) ← final answer
```

Every intermediate number is **output through the analytical head and fed back through the analytical encoder**. No digit manipulation at any step. Each step is a simpler arithmetic operation that the transformer can handle in one forward pass.

The key insight: with number embeddings on both sides, **the transformer never works with digits**. It works with continuous numerical values. Addition of two numbers becomes a geometric operation in embedding space, not a string manipulation algorithm.

Pros and cons — honest assessment

Pros

1. **End-to-end numerical.** No digit tokenization anywhere. Numbers stay as continuous values from input to output.
2. **Magnitude is free.** The model never has to learn that "1000" is bigger than "999" — the embeddings encode this structurally.
3. **Constant computational cost per number.** "7" and "7000000" are both one embedding. No variable token count.
4. **Chain-of-thought becomes numerical, not textual.** Each intermediate step passes actual values, not digit strings. Errors compound less because there's no digit-assembly noise.
5. **The encoder is analytically exact.** No training needed for input encoding. The structure is mathematical, not learned.
6. **The output decode is analytically exact.** `exp(log_mag)` and `sign()` introduce zero approximation error. The only error is in the transformer's prediction of what number to output — which is the model's job.
7. **Differentiable end-to-end.** Gradients flow from the output number loss through the analytical decode, through the transformer, into the number embeddings. The whole system trains jointly.

Cons

1. **Pretraining from scratch.** You can't retrofit this into GPT-4. Every weight in the model was trained against text token embeddings. You need a new pretraining run with the hybrid tokenizer — enormous compute cost.
2. **Dual-mode generation is hard.** The model must learn when to use the text head vs the number head. Misrouting (outputting "five" as text when it should output `embed(5)`) creates training instability. This is a non-trivial control flow problem in autoregressive generation.

3. **Precision is bounded by the transformer, not the embedding.** Even with perfect analytical I/O, the transformer must still *compute* the right answer internally. For 347×892 , the model must learn that multiplication maps to a specific function in embedding space. Whether it can learn this depends on model capacity and training data, not on the encoder.
4. **Exact integer arithmetic is still hard.** $347 \times 892 = 309524$, not 309523 or 309525. The number head outputs a continuous value. Even a tiny error (309524.3) is "wrong" for integer arithmetic. You'd need a rounding step, and knowing *when* to round (integers yes, pi no) requires the model to understand number types.
5. **Training data format changes.** All existing text corpora have numbers as text. You'd need a preprocessing pipeline that detects numbers, converts them to embeddings, and handles edge cases (dates, phone numbers, ZIP codes, version numbers like "3.11" — which are not the number 3.11).
6. **Variable-precision requirements.** Financial: exact to 2 decimal places. Scientific: significant figures. Integer: exact. The embedding treats them all the same. You might need type-aware output heads.
7. **No existing ecosystem.** Tokenizers, training frameworks, evaluation benchmarks — everything assumes text tokens. You'd be building infrastructure from scratch.

Suggested path forward

Phase 1: Prove the encoder helps. Take a small transformer (GPT-2 scale). Train two versions on a math dataset: one with standard tokenization, one with number embeddings on input + text token output. Compare on numerical reasoning tasks (comparison, estimation, magnitude). This isolates the encoder's value without the output head complexity. Cheap to run, publishable result.

Phase 2: Add the number output head. Same small transformer, now with the dual-head architecture. Train on arithmetic tasks. The analytical output head is just a linear projection + exp/sign — trivially implementable. Compare against the text-output version. This proves the end-to-end concept.

Phase 3: Scale. If phases 1–2 show clear gains, pretrain a larger model with the full hybrid architecture. This is the expensive step, but by this point there would be evidence to justify it.

For the analytical output head specifically — the implementation in PyTorch would be:

```
# Number head: linear projection from transformer hidden state
number_proj = nn.Linear(d_model, 2) # outputs (log_mag, sign_logit)

# At inference:
log_mag, sign_logit = number_proj(hidden_state).split(1, dim=-1)
x_hat = torch.sign(sign_logit) * torch.exp(torch.clamp(log_mag, -14, 14))

# At training:
# Loss directly on x_hat vs target number -- gradients flow through exp and sign
```

The analytical output head is 3 lines of code. The hard part is the transformer learning what number to produce — which is the model's job, not the embedding's.

Revised bottom line

The project isn't about "numerical literacy" — it's about **replacing text-based number processing with continuous numerical processing end-to-end**. The encoder is the input side. The output side is a linear projection + analytical decode. The transformer in the middle learns numerical computation in embedding space instead of string manipulation in token space.

This won't make transformers into calculators for arbitrary-precision arithmetic. But it removes the floor that text tokenization imposes on numerical reasoning. LLMs are bad at math for two reasons: (1) bad representation (tokenization) and (2) limited computation depth. This project fixes reason 1 completely. Reason 2 remains, but it's a smaller problem when the representation is right.